MPI Parallelization of MLP Training on MNIST

Final Project Report

Author One* Author Two* Author Three
[Course Name & Number, Institution]

May 2026

GitHub Repository: github.com/your-org/your-repo

Abstract

We study how far MPI-based parallelism can accelerate end-to-end neural-network training while preserving training quality under controlled settings. Starting from a serial C++17 MLP baseline on MNIST, we implement and compare four MPI execution modes: flat data parallelism, two-level hierarchical data parallelism, local-SGD with periodic synchronization, and layer-wise model parallelism. To keep comparisons fair, all backends use matched model shape, optimizer configuration, dataset subsets, and output metrics schema, and are launched with one thread per rank for BLAS/OpenMP runtimes. We also apply shared engineering optimizations such as memory reuse in hot paths and release compiler settings. The main empirical outcome is that communication structure, not only rank count, determines realized speedup: local-SGD gives the best raw epoch-time scaling at large cluster size, flat DP remains a strong and stable baseline, hierarchical DP does not outperform flat DP for the current model size, and model parallelism is most sensitive to partition balance and communication boundaries.

1 Hypothesis

We investigate whether communication-aware MPI training strategies can deliver better time-to-epoch than a standard flat all-reduce baseline while maintaining comparable optimization behavior on MNIST. Our hypothesis has two parts:

- communication pattern matters as much as rank count, so methods that reduce or amortize global synchronization should scale better at larger process counts;
- implementation details (memory reuse, consistent launch semantics, and stable timing definitions) are necessary to expose the true algorithmic differences across backends.

The observed results support this hypothesis. Local-SGD achieves the best large-scale epoch-time reduction, flat DP remains the strongest fully synchronous baseline, and hierarchical DP underperforms flat DP in our current workload regime because its extra synchronization phases are not sufficiently amortized by model size.

2 Context

Member contributions.

*Equal contribution.

- **Author One (replace with final name):** serial baseline, shared training utilities, and flat data-parallel baseline.
- **Author Two (replace with final name):** hierarchical DP and local-SGD variants, launch scripts, and scaling experiments.
- **Author Three (replace with final name):** model-parallel path, reporting pipeline, and methodology/results write-up.

Broader context. This project is a course-focused systems study rather than a direct sub-project of an external research lab artifact. Its emphasis is reproducible performance evaluation of parallel training designs under matched conditions.

Applicability to other courses. Variants of this project are immediately reusable in high-performance computing, distributed systems, and machine-learning systems courses because the same codebase supports serial baselining, collective-communication studies, point-to-point pipeline studies, and scaling-law analysis with consistent metrics outputs.

3 Prior Work

Work 1. Sergeev and Del Balso [2018] established a practical architecture for synchronous distributed data-parallel training that relies on collective communication and minimal intrusion into the training loop. Our flat-DP baseline follows this same principle: local gradient computation plus global aggregation each step, with careful control of launch and threading semantics.

Work 2. Stich [2019] showed that periodic synchronization can significantly reduce communication while retaining useful convergence behavior. Our local-SGD mode adopts this idea by synchronizing fused model weights every K steps and quantifying the resulting speed/accuracy trade-off in a controlled MPI setting.

Work 3. Narayanan et al. [2019] demonstrated the importance of pipeline-aware scheduling in model-parallel training and highlighted bubble overhead as a first-order performance factor. Our current model-parallel implementation is non-pipelined but includes explicit pipeline placeholders and an alpha-beta communication model to prepare a clean extension path for GPipe/1F1B-style variants in Section 4.

4 Empirical Methodology

4.1 Problem Setup

We study parallel training of a feed-forward MLP on MNIST with a fixed optimization setup and matched data split across implementations. The primary model for scaling is $784 \to 256 \to 128 \to 64 \to 10$, trained with SGD and cross-entropy. The project compares five execution modes:

- **Serial**: single-process baseline.
- **Flat MPI Data Parallel (DP)**: each rank computes local gradients, then performs gradient all-reduce every step.
- **Hierarchical MPI DP**: per-step gradient synchronization split into intra-node reduce, inter-node all-reduce (leaders), and intra-node broadcast.
- **Local SGD**: each rank performs local SGD and averages weights every K steps.

- **Model Parallel (MP)**: layers are partitioned across ranks and activations/gradients are exchanged between neighboring partitions.

4.2 Sequential Baseline

The sequential baseline uses the same MLP, loss, and optimizer as all MPI variants. It runs a shuffled mini-batch loop over training data and reports `epoch_time_ms` as *training-loop-only* wall time (validation evaluation is measured but excluded from epoch timing). This timing definition is intentionally mirrored in the MPI backends to keep scaling comparisons consistent.

4.3 Parallel Design

Why multiple algorithms are shown. The implementations do not share one execution pattern. Serial training has no communication. Data-parallel methods synchronize gradients or weights with collectives. Model parallelism partitions layers and exchanges intermediate tensors point-to-point. For clarity, we therefore present separate algorithms for (1) serial training, (2) data-parallel training, and (3) model-parallel training.

Serial. Single-process SGD over shuffled mini-batches; no MPI calls.

Algorithm 1 Serial training epoch

Require: shuffled indices I, batch size B, parameters θ
Ensure: updated θ after one epoch
1: **for** $t = 0, B, 2B, \ldots$ **do**
2: gather batch (x, y) from I
3: $g \leftarrow \nabla \ell(\theta; x, y)$
4: $\theta \leftarrow \theta - \eta g$
5: **end for**

Flat DP. Each rank processes a disjoint shard of each global batch, computes local gradients, then uses `MPI_Allreduce` on layer gradients to form the global mean gradient before applying the update.

Hierarchical DP. Gradient synchronization is decomposed into three phases: `MPI_Reduce` within a node, `MPI_Allreduce` across node leaders, then `MPI_Bcast` within each node. This reduces inter-node participants but introduces extra synchronization phases.

Local SGD. Ranks execute local updates independently and synchronize model weights via one fused `MPI_Allreduce` every K steps ($K \in \{10, 50\}$ in this report). This trades statistical freshness for lower communication frequency.

Model Parallel (MP). Layers are partitioned across ranks. Forward activations and backward gradients are exchanged via point-to-point MPI between adjacent layer partitions. In this report, MP is treated as a first-class method in the main project (not an auxiliary baseline). We currently evaluate a non-pipelined schedule, and we reserve explicit placeholders for pipelined MP variants:

- **Pipeline placeholder A (inference-style fill/drain):** TODO add timeline and throughput model for micro-batch pipeline.

- **Pipeline placeholder B (1F1B schedule):** TODO add 1F1B warmup/steady/cooldown schedule and bubble analysis.
- **Pipeline placeholder C (interleaved virtual stages):** TODO add interleaved stage assignment and expected memory trade-offs.

Algorithm 2 Data-parallel epoch (flat DP / hierarchical DP / local SGD)

Require: shuffled index list I, global batch size B, rank count P
Ensure: updated model parameters after one epoch

for global batch offset $t = 0, B, 2B, \ldots$ **do**
  gather local mini-batch (x_r, y_r) from I
  compute local gradients $g_r \leftarrow \nabla \ell(\theta; x_r, y_r)$
  if mode is Flat DP **then**
    $g \leftarrow$ `AllreduceMean`(g_r)
  else if mode is Hierarchical DP **then**
    $g \leftarrow$ `IntraReduce` $\rightarrow$ `InterAllreduce` $\rightarrow$ `IntraBcast`
  else if mode is Local SGD **then**
    set $g \leftarrow g_r$; synchronize weights only every K steps
  end if
  apply SGD update with synchronized quantity (gradient or weight)
end for

Algorithm 3 Model-parallel epoch (non-pipelined, per-batch)

Require: layer partition per rank, batch size B
Ensure: updated local layer parameters on each rank

for each batch (x, y) **do**
  receive activation from previous rank (or use input if first rank)
  compute local forward over owned layers
  send activation to next rank (unless last rank)
  receive backward gradient from next rank (or compute from loss if last rank)
  compute local backward over owned layers
  send gradient to previous rank (unless first rank)
  update local parameters
end for

4.4 Communication Cost Model (Alpha-Beta)

We use the standard latency-bandwidth model:

$$T_{\text{msg}}(m) = \alpha + \beta m, \tag{1}$$

where α is startup latency, β is per-byte (or per-float) transfer cost, and m is message size.

Flat DP (per step). With all-reduce payload m_g, a ring-style approximation is:

$$T_{\text{flat}}(m_g) \approx 2(P-1)\alpha + 2\frac{P-1}{P}\beta m_g. \tag{2}$$

Hierarchical DP (per step). Let q be ranks per node and N be number of nodes ($P = Nq$). A simple two-level approximation (intra reduce+bcast, inter all-reduce among leaders) is:

$$T_{\text{hier}}(m_g) \approx \underbrace{2\log q\,\alpha + 2\frac{q-1}{q}\beta m_g}_{\text{intra-node}} + \underbrace{2(N-1)\alpha + 2\frac{N-1}{N}\beta m_g}_{\text{inter-node}}. \tag{3}$$

This can outperform flat DP when expensive inter-node traffic is reduced and message size is large enough to amortize extra phases.

Local SGD (amortized per step). If weight synchronization payload is m_w and synchronization is every K steps:

$$T_{\text{local,step}} \approx T_{\text{compute}} + \frac{1}{K}T_{\text{sync}}(m_w), \tag{4}$$

which explains reduced communication overhead as K increases.

Model Parallel (non-pipelined). If activation payload per boundary is m_a, backward payload is m_b, and rank r owns local compute C_r:

$$T_{\text{mp,step}} \approx \sum_{r=1}^{P} \left(C_r + (\alpha + \beta m_a) + (\alpha + \beta m_b)\right). \tag{5}$$

The practical bottleneck is the slowest stage plus boundary communication.

Pipeline placeholders (to be completed experimentally). For L pipeline stages and M microbatches, fill-drain bubble fraction is:

$$\text{bubble}(L, M) = \frac{L-1}{M+L-1}. \tag{6}$$

Planned additions:

- GPipe (fill-drain) throughput model and measured bubble.
- 1F1B steady-state model with overlap assumptions.
- Interleaved stage model with memory/throughput trade-offs.

4.5 Implementation Details

The system is implemented in C++17 with MPI collectives/point-to-point communication and BLAS-backed dense linear algebra. Key implementation choices that affect performance and reproducibility are:

- **BLAS acceleration:** matrix multiply and vector updates were moved to BLAS-backed kernels (`sgemm/saxpy/sgemv`) to reduce scalar-loop overhead.
- **Backend isolation:** serial, flat DP, hierarchical DP, local-SGD, and model-parallel paths are built as separate executables with dedicated launch scripts.
- **Memory reuse in hot loops:**
  - preallocated mini-batch feature/label buffers are reused across steps to avoid per-batch allocation churn;
  - gather paths use contiguous writes to improve locality;
  - gradient and workspace buffers are reused in the shared MLP training path.

- **Compiler configuration:** Release builds use `-O3`, with optional `-march=native` and optional IPO/LTO when supported.

Because model parallelism uses a separate `MPLayerSlice` forward/backward path, some MLP-internal workspace optimizations apply directly to serial and data-parallel modes, while MP performance is driven more by partitioning balance and point-to-point communication behavior.

4.6 Experimental Setup

All strong-scaling sweeps were executed through Slurm `srun` launchers. Thread-level oversubscription was disabled by pinning: `OMP_NUM_THREADS=1`, `OPENBLAS_NUM_THREADS=1`, `MKL_NUM_THREADS=1`. The primary sweep used:

- epochs = 10, global batch = 1024
- train/validation samples = 50,000/10,000
- learning rate = 0.03, seed = 42
- hidden layers = $256, 128, 64$
- rank ladder: $1n4r \rightarrow 1n16r \rightarrow 1n32r \rightarrow 2n64r \rightarrow 4n128r$

Timing and summary outputs are generated from `results/scaling/*.csv` and `results/scaling/*.log` via `summarize_dp_scaling.py` and `make_scaling_visuals.py`.

Table 1: Software/runtime configuration used for scaling runs.

Component	**Specification**
Runtime launcher	Slurm `srun` + MPI
Threading policy	OMP/BLAS/MKL threads pinned to 1
Build mode	Release (`-O3`, optional `-march=native`, optional IPO)
Dataset	MNIST (local cache via `prepare_mnist.sh`)
Model for main sweep	MLP $256, 128, 64$ hidden layers
Scaling configs	$1n4r, 1n16r, 1n32r, 2n64r, 4n128r$

4.7 Results

4.7.1 Scaling Analysis

Figure 1 shows epoch time vs cluster size with serial baseline and ideal scaling reference. Local-SGD achieves the best time at 128 ranks (K=50 fastest), while flat DP remains more efficient than hierarchical DP for this model size and workload regime. This indicates that hierarchical communication overhead (extra phases per step) is not amortized sufficiently at the current gradient payload.

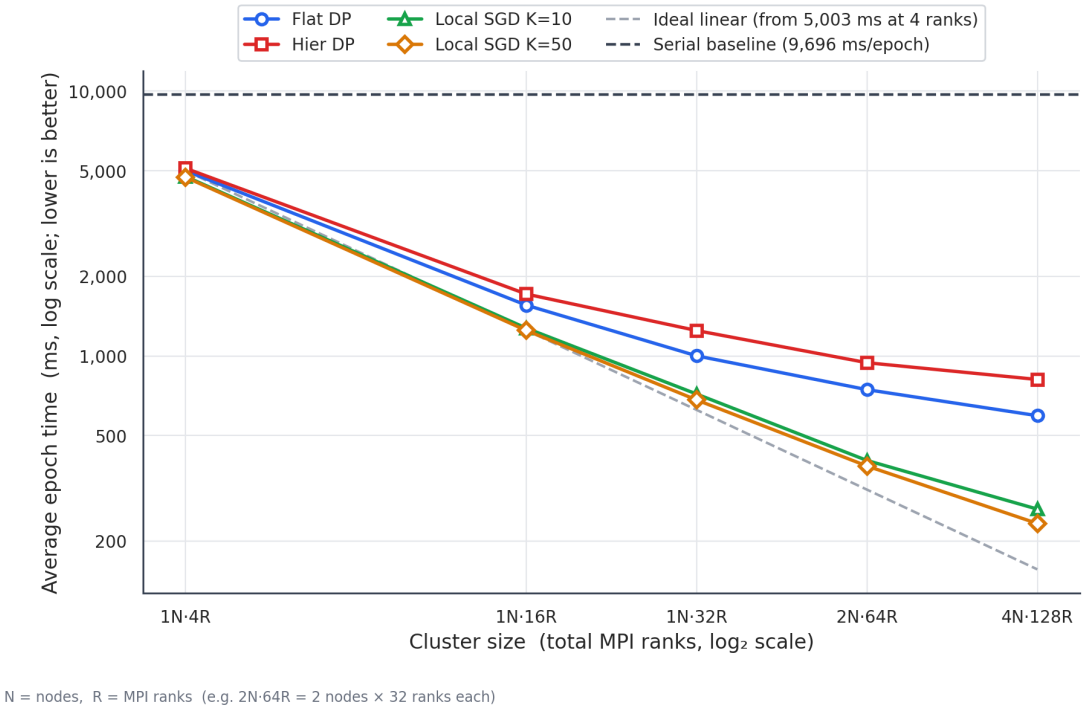

Figure 1: Epoch time vs cluster size with serial baseline and ideal line (MLP 256 → 128 → 64, MNIST, 10 epochs).

4.7.2 Strong Scaling

Using flat DP at $1n4r$ as baseline ($\approx$ 5003 ms/epoch), strong-scaling speedup and efficiency are:

Table 2: Strong scaling for flat DP (baseline: $1n4r$).

Config	**Time (ms)**	**Speedup** $S(p)$	**Efficiency** $E(p)$
1n4r	5003	1.00×	100.0%
1n16r	1555	3.22×	80.4%
1n32r	1001	5.00×	62.5%
2n64r	746	6.71×	41.9%
4n128r	595	8.40×	26.3%

Compared to flat DP, hierarchical DP trails at all tested scales in the current runs, while Local-SGD surpasses flat DP speedup as K increases, reaching 21.46× speedup at $4n128r$ (K=50) relative to the flat-DP 4-rank baseline.

4.7.3 Weak Scaling

This report focuses on strong scaling. A dedicated weak-scaling sweep (proportionally increasing data size with rank count) is not yet included in the current artifact set and is left as future work.

4.7.4 Additional Results

Timing breakdowns from logs show:

- flat DP: communication cost appears primarily in `allreduce_ms`, while `other_ms` remains non-negligible at high ranks.
- hierarchical DP: cost is split into `intra_reduce_ms`, `inter_allreduce_ms`, and `intra_bcast_ms`, confirming additional synchronization phases.
- local-SGD: `sync_ms` is much smaller than per-step DP comm cost because communication occurs only every K steps.

Accuracy trade-off is modest for the fastest local-SGD settings in these runs: at 128 ranks, K=50 is fastest with a small validation-accuracy drop relative to flat DP.

5 Challenges and Obstacles

[Challenge 1]. TODO: describe what you tried, what happened, and your hypothesis for why.

[Challenge 2]. TODO.

[Challenge 3]. TODO.

References

Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil Devanur, Gregory Ganger, Phillip Gibbons, and Matei Zaharia. PipeDream: Fast and efficient pipeline parallel DNN training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 2019.

Alexander Sergeev and Mike Del Balso. Horovod: Fast and easy distributed deep learning in TensorFlow. In *Proceedings of the 31st Conference on Neural Information Processing Systems Workshop*, 2018.

Sebastian U. Stich. Local SGD converges fast and communicates little. In *International Conference on Learning Representations*, 2019.